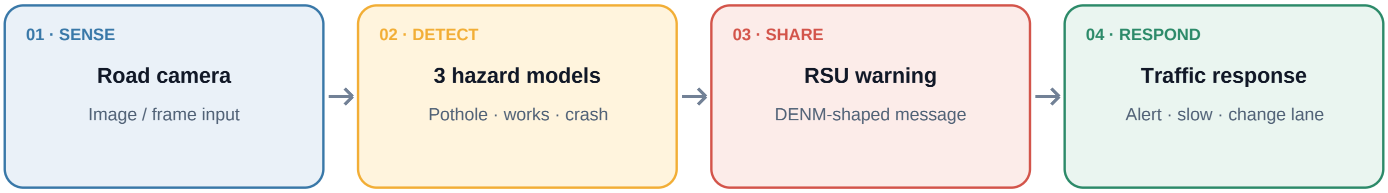
PROJECT BRIEF / EDGE AI × COOPERATIVE MOBILITY

See a road hazard.
Warn the next vehicle.

A project guide to the models, edge-inference work, and SUMO simulations in PFPD-II

THE BUILD	THE CURRENT BOUNDARY
Three focused road-hazard detectors feed an edge-to-vehicle warning concept, with SUMO used to explore how earlier warnings could change traffic behavior.	Model files and simulation code are in the repo. Edge speed, full live inference in the corridor, and real vehicle/V2X hardware behavior still need validation.

The idea in one loop



01 · SENSE

Road camera

Image / frame input

→

02 · DETECT

3 hazard models

Pothole · works · crash

→

03 · SHARE

RSU warning

DENM-shaped message

→

04 · RESPOND

Traffic response

Alert · slow · change lane

This is a cooperative safety concept: a vehicle or roadside camera detects a hazard, an edge device turns it into a compact warning, and other vehicles can react before reaching the same location. The repository brings together model artifacts, inference code, message construction, and SUMO traffic scenarios to develop and study that loop.

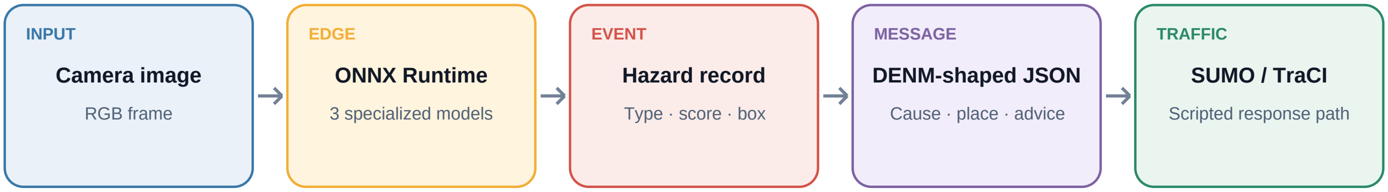
What is in the project today

MODEL ARTIFACTS	EDGE CODE	TRAFFIC STUDY
Three YOLO11s INT8 ONNX files in `quantization/`, each about 9.48 MB.	ONNX Runtime inference and DENM-shaped message code in `simulation/scripts/`.	A 1 km SUMO corridor with two signalized intersections and three placed hazard zones.

01 / SYSTEM ARCHITECTURE

From a camera frame to a traffic warning

The repository has several working pieces. The diagram shows the intended system path and labels the boundaries between implemented code and the simulation scenario.



INPUT
Camera image
RGB frame

EDGE
ONNX Runtime
3 specialized models

EVENT
Hazard record
Type · score · box

MESSAGE
DENM-shaped JSON
Cause · place · advice

TRAFFIC
SUMO / TraCI
Scripted response path

Three specialists

The model set divides perception into pothole / damaged surface, construction / barricades, and accident / debris. The quantized files are present under `quantization/`. The code in `simulation/scripts/rsu_hazard_disseminator.py` contains an ONNX Runtime detector and a DENM-shaped event builder.

Two separate demonstrations

Component	What its code does	Reading it accurately
Image-to-event script	Loads an ONNX model when dependencies and files are av[illegible]	[illegible]
1 km cooperative SUMO ru[illegible]	Places three hazards on a corridor. The first vehicle encount[illegible]	[illegible]

The pieces point toward one integrated Edge AI → RSU → connected-vehicle system. Current repository evidence supports a prototype made of inference/message code plus a SUMO cooperative-response demonstration; it does not yet establish a camera-to-radio deployment running end to end.

02 / MODEL SUITE AND EVIDENCE

Three edge artifacts, with performance claims kept in scope

The project records two different model families. Keeping their results separate avoids attributing YOLO11n evaluation numbers to the YOLO11s quantized models.

Recorded YOLO11s suite

Quantized YOLO11s artifact	Size	Recorded reference P / R / mAP50	Recorded CPU result
pothole_yolo11s_int8.onnx	9.48 MB	93.11% / 89.45% / 95.64%	2,868 ms mean 0.3 FPS
construction_yolo11s_int8.onnx	9.48 MB	95.20% / 91.10% / 92.70%	2,998 ms mean 0.3 FPS
accident_obstacle_yolo11s_int8.onnx	9.48 MB	96.50% / 90.80% / 92.47%	3,040 ms mean 0.3 FPS

The precision / recall / mAP figures above are copied from the GPU-reference constants in `results/empirical_model_comparison.json` and `src/fast_empirical_benchmark.py`. The same result file records the CPU timings shown here. The local checkout does not include matching YOLO11s test-split reports to independently verify those reference accuracy figures. Treat them as project-reported metrics, not an audited comparison.

Separate local YOLO11n evaluation records

Recorded experiment	Test images	Precision	Recall	mAP50	mAP50–95	INT8 file
Pothole · 7,066-image report	708	86.18%	82.77%	89.32%	61.19%	2.87 MB
Construction · YOLO11n report	122	89.84%	77.33%	89.20%	60.71%	2.87 MB

These are YOLO11n records in `model_training/`, not the three YOLO11s quantization files. The pothole report has 708 held-out images; the construction report has 122. No corresponding accident-model evaluation summary was found in the local report files.

What the edge timing says

The recorded CPU means for the three YOLO11s ONNX models are 2,868–3,040 ms per inference (about 0.3 FPS in the benchmark record). That measurement does not support the README's separate ~35 FPS Raspberry Pi claim. Hardware, runtime, preprocessing, and measurement method must be pinned down before making a real-time edge-performance claim.

The profile loads three models concurrently into memory; process RAM is 107.9 MB including about 54.0 MB reported runtime overhead. This is a software benchmark record, not Raspberry Pi board validation.

03 / SUMO COOPERATIVE SIMULATION

Study how a warning could change what traffic does

The corridor scenario makes the cooperative idea visible: one vehicle encounters a hazard first, the RSU remembers it, and later vehicles are warned upstream.

CORRIDOR
1,000 m
Two signalized junctions

→

DETECTOR
First vehicle
Discovers scenario event

→

RSU
Stores event
DENM-shaped packet

→

RECEIVERS
Following cars
Lane / speed response

Scenario item	Repository configuration
Road	1 km arterial corridor; two signalized intersections; two mainline lanes.
Roadside unit	RSU_ZONE_01 at approximately x = 500 m.
Hazard zones	Construction at about 550 m; pothole at about 650 m; accident / debris at about 730 m.
Traffic	Continuous connected-car flow plus opposing and cross-street traffic in the route file.
Message fields	Hazard type, ETSI cause-code mapping, location, severity, recommended action, and advisory speed are represented in JSON structu
Response	TraCI script changes lane and speed in the scenario; other code paths attempt lane restrictions or speed limits.

How to explain the demo

SUMO is a traffic simulator. It lets the team test traffic behavior and visualize hazard-warning logic on a defined road network. In the current 1 km runner, detector/receiver roles and vehicle reactions are rule-based. They illustrate the dissemination concept; they are not evidence that physical V2X radios, production DENM interoperability, or a live camera inference stream has been validated.

There is also a separate image inference / TraCI path in `simulation/scripts/rsu_hazard_disseminator.py`. Treat that path and the corridor scenario as related prototype components, not as one proven integrated deployment.

Primary files

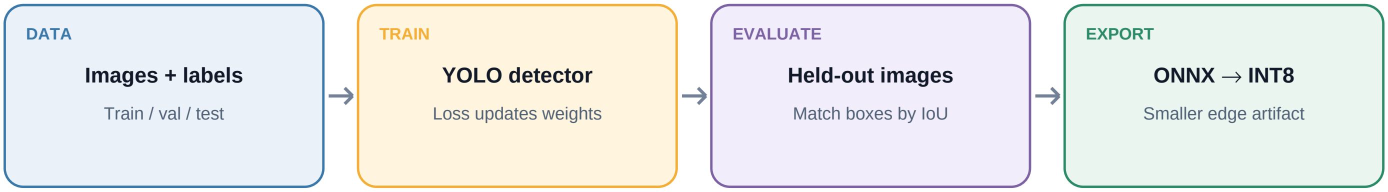
`simulation/corridor_1km/run_cooperative_v2x_sim.py` · scenario loop
`simulation/corridor_1km/corridor.sumocfg` and `corridor.rou.xml` · network timing and flows
`simulation/scripts/rsu_hazard_disseminator.py` · ONNX inference and warning-message code

04 / TRAINING AND PROJECT STATE

A useful baseline for the next build phase

This section summarizes how the project trains and scores a detector, while tying project-specific results to the reports that are actually present.

How a detector is trained and scored



Precision answers: of the boxes the model predicted, how many were correct? Recall answers: of the labeled hazards, how many did it find? AP summarizes a precision–recall curve for a class. mAP@50 averages AP using an IoU match threshold of 0.50; mAP@50–95 averages over stricter IoU thresholds too. A strong score on one frozen test set does not establish field performance or device speed.

Where the project stands

READY TO SHOW	NEEDS THE NEXT EVIDENCE
Three quantized model artifacts; training and export scripts; per-domain YOLO11n reports for pothole and construction; multi-hazard SUMO corridor and warning logic.	Reproducible YOLO11s test-set reports for all three files; calibrated INT8-vs-FP32 accuracy; target-board latency; live camera-to-inference-to-RSU flow; radio/network interoperability; controlled safety/traffic evaluation.

Practical next sequence

1. Freeze the three YOLO11s datasets, splits, checkpoints and evaluation commands.
2. Re-run per-class FP32 and INT8 validation on the same held-out images.
3. Benchmark each model and the three-model pipeline on the intended edge board, recording preprocessing and end-to-end latency.
4. Connect live image detections to one hazard-event schema; then connect that message path to a controlled SUMO experiment.
5. Compare warned and unwarned traffic outcomes before making safety or deployment claims.

Repository map

`quantization/` · three YOLO11s INT8 files & model note | `model_training/` · training, export, quantization and YOLO11n reports
`results/` · recorded three-model benchmark profile | `simulation/` · SUMO networks, corridor scenario and dissemination code

Prepared from the PFPD-II working tree and the linked GitHub repository. Metric provenance and implementation boundaries are called out so the guide can introduce the project without overstating validation.